

Queues as Linked Lists in C

The C function to **insert an element** into a queue is given as follows:

Let `front` and `rear` be two global pointers pointing to the **first node** and **last node** of the queue respectively.

```
void insert(int x)
{
    NODEPTR p;
    p = getnode();
    p->info = ele;
    p->next = NULL;

    if (rear == NULL)
    {
        front = p;
    }
    else
    {
        Rear->next = p;
    }
    rear = p;
}
```

Explanation:

- `NODEPTR` is a pointer to a node structure (e.g., `typedef struct node* NODEPTR;`).
- `getnode()` is a function that allocates memory for a new node.
- `info` stores the element to be inserted.
- `next` is the link to the next node.
- If the queue is empty (`rear == NULL`), both `front` and `rear` point to the new node.
- Otherwise, the new node is linked at the end, and `rear` is updated to point to it.

Queue as Linked List in C

Structure Definition

```
struct node {
    int info;
    struct node *next;
};
```

```
typedef struct node* NODEPTR;

NODEPTR front = NULL, rear = NULL;
```

Function to Get a New Node

```
NODEPTR getnode() {
    NODEPTR p;
    p = (NODEPTR)malloc(sizeof(struct node));
    return p;
}
```

Insert Function

```
void insert(int x)
{
    NODEPTR p;
    p = getnode();
    p->info = x;
    p->next = NULL;

    if (rear == NULL)    // Queue is empty
    {
        front = p;
    }
    else
    {
        rear->next = p; // Link new node at end
    }
    rear = p;           // Move rear to new node
}
```

Delete Function

```
int delete()
{
    NODEPTR p;
    int x;

    if (front == NULL)
    {
        printf("Queue is empty\n");
        return -1;
    }

    p = front;
    x = p->info;
    front = front->next;

    if (front == NULL)    // Queue becomes empty
        rear = NULL;

    free(p);
    return x;
}
```

Display Function

```
void display()
{
    NODEPTR p;
    if (front == NULL)
    {
        printf("Queue is empty\n");
        return;
    }

    p = front;
    printf("Queue elements: ");
    while (p != NULL)
    {
        printf("%d ", p->info);
        p = p->next;
    }
    printf("\n");
}
```

C Function to Remove an Element from a Queue

```
void remove()
{
    int x;
    NODEPTR p;

    if (front == NULL)
        printf("Queue underflow\n");
    else
    {
        p = front;
        ele = p->info;
        front = p->next;

        if (front == NULL)
            rear = NULL;

        free(p);
        printf("Removed element = %d\n", x);
    }
}
```

Explanation:

- The function checks if the queue is empty (`front == NULL`).
 - If true, it prints "Queue underflow".
- Otherwise:

- The first node (`front`) is stored in `p`.
- The element to be deleted is taken from `p->info`.
- `front` moves to the next node (`front = p->next`).
- If the queue becomes empty (`front == NULL`), `rear` is also set to `NULL`.
- Finally, the deleted node is freed using `free(p)`.
- The removed element is displayed.